Temporal ChatKBQA: From ChatKBQA Baseline to a Submission-Ready NLP System

Bao-Kiet Truong

Student ID: 22127223

Ho Chi Minh University of Science
Faculty of Information Technology
Application of NLP in Industry

26/05/2026

Business Problem and Stakeholders

Problem

Users need answers to time-aware factual questions, but Freebase requires schema knowledge, entity IDs, relation IDs, and SPARQL expertise (Bollacker et al. 2008).

Example questions

- Who was US president before Obama?
- What was the first Harry Potter novel?
- Where did an artist live at a given time?

Stakeholders

- analysts and researchers
- BI and compliance teams
- knowledge-graph engineers

Submission framing

This is a production-oriented NLP system artifact, not a research-only notebook.

Why Temporal KBQA Needs NLP

NLP task

Map free-form temporal language into executable KB structure:

```
Question -> S-expression -> SPARQL -> Answer
```

Success metrics

- F1 / Hits@1 / Accuracy
- valid S-expression rate
- answer rate and executor errors
- demo/API readiness

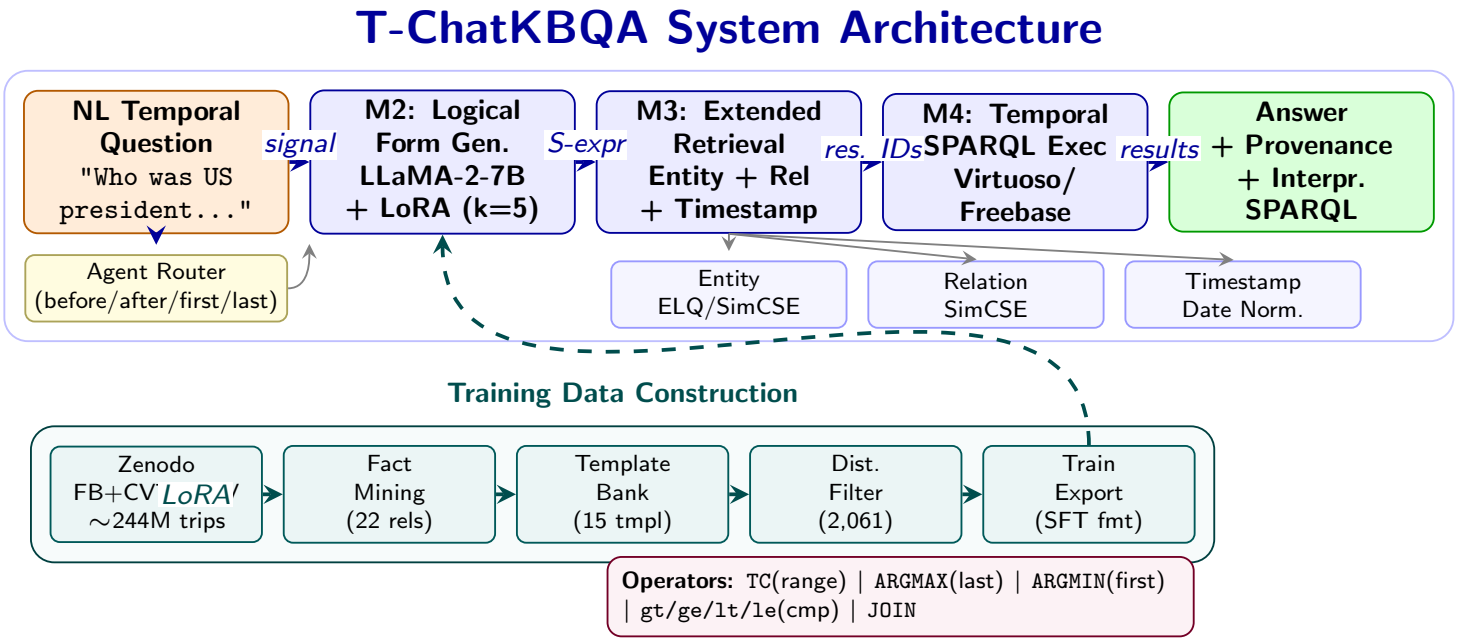
Temporal signals

before, after, during, first, last, earliest, latest, now

Current result framing

The v1 system is submission-ready as an engineering artifact, but not performance-ready as a production QA model.

T-ChatKBQA System Architecture



Key design decisions

- ChatKBQA `generate-then-retrieve` core preserved
- Temporal operators: TC, ARGMAX, ARGMIN, gt/ge/lt/le
- Zenodo FB+CVT+REV replaces corrupt Virtuoso dump
- 22 temporal relations mined, 15 templates, 2,061 synthetic samples
- LLaMA-2-7B + LoRA (q_proj, v_proj), 5 epochs, 645 steps
- Agent router separates temporal vs. standard KBQA

Production-Oriented Repository

Implemented surfaces

- FastAPI API
- CLI entrypoints
- Streamlit demo layer
- Dockerfile
- targeted tests

Reproducibility tooling

- DVC pipeline
- Makefile shortcuts
- YAML config validator
- evaluation harness
- dataset download helpers

Why it matters

The project can be inspected, run in degraded demo mode, packaged, and evaluated through repeatable commands rather than private notebooks.

Data Sources, License, and Split

Sources

- TempQuestions: English temporal QA benchmark, 1,271 questions (Jia et al. 2018)
- Freebase: archived KB, CC-BY data dump (Bollacker et al. 2008)
- idirlab/freebases: CC0-1.0 processed mirror used for v1 facts
- FACC1/ELQ-style assets: entity retrieval support, not redistributed

Locked split policy

- canonical TempQuestions: 888 train / 383 test
- v1 synthetic training pool: 2,061 examples
- tuning split: 1,855 train / 206 validation
- final evaluation artifact: 268 human test questions

Data reality

The bundled human TempQuestions path is partially corrupted, so v1 training uses filtered synthetic temporal supervision and keeps the human test artifact held out.

Data Pipeline and Cleaning

Pipeline

1. audit TempQuestions and merged artifacts
2. mine / normalize temporal facts
3. generate synthetic temporal samples
4. filter by quality and operator distribution
5. export ChatKBQA instruction-tuning format

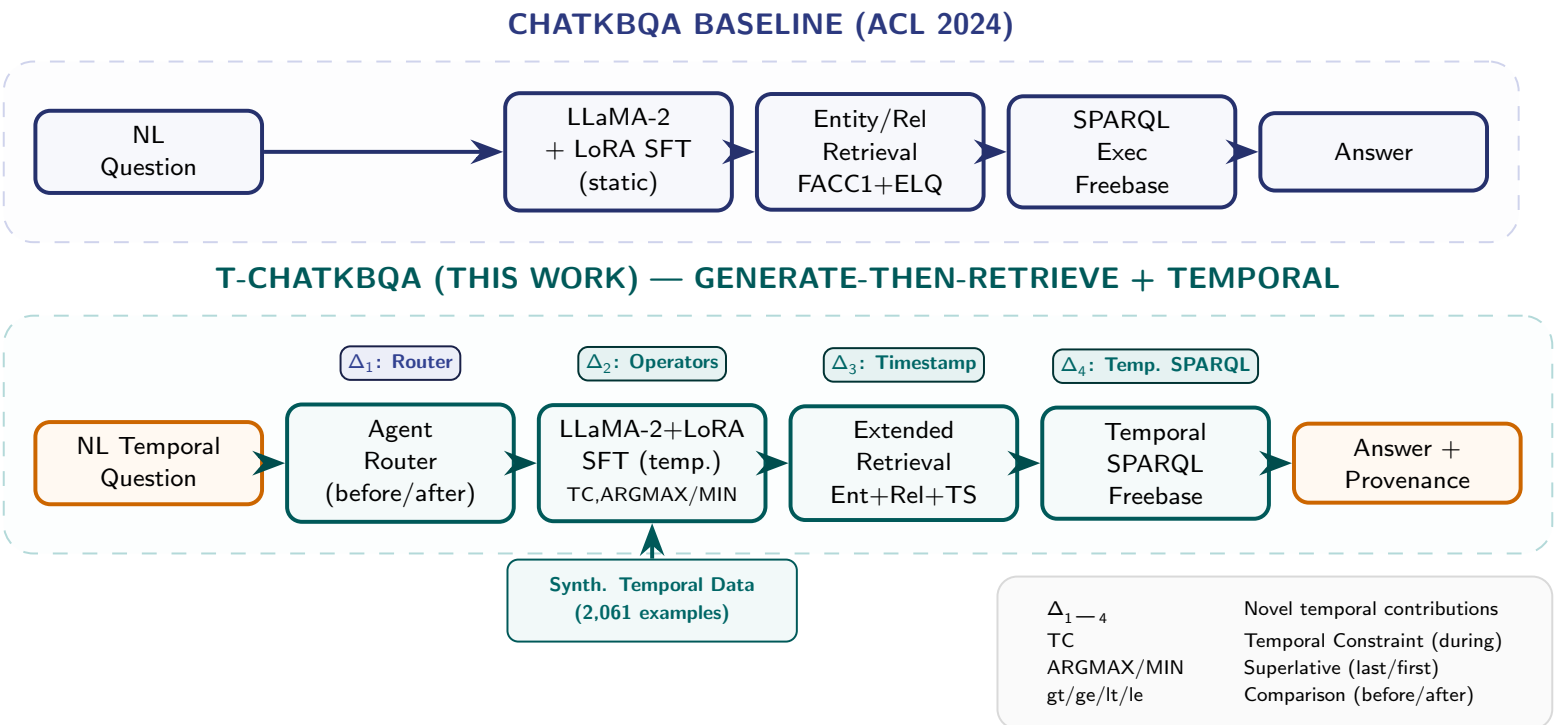
Cleaning targets

- placeholder-collapsed SPARQL
- suspicious merged records
- missing date literals
- noisy relation patterns

Design principle

Treat data quality as a first-class engineering artifact, not as hidden preprocessing.

T-ChatKBQA vs. ChatKBQA Baseline



Model Selection and Tuning

Chosen model

- LLaMA-2-7B generator (Touvron et al. 2023)
- LoRA adaptation (Hu et al. 2022)
- ChatKBQA-compatible S-expression output

Basic tuning

Compared 3/5/10 epochs, 3e-5/5e-5/1e-4 learning rates, and beam 3/5/8 using the synthetic validation split. The selected run balanced syntax validity, stability, and runtime.

Final config

5 epochs, 5e-5 learning rate, effective batch 16, beam size 5, max 256 new tokens.

Verified run artifact

Training completed for 645 steps on 2,061 synthetic examples; reported values follow run artifacts, not stale template defaults.

Evaluation Setup and Baselines

Method	Same Dataset?	Role	Why included
Generate-only	Yes	Main baseline	Measures raw temporal generator without effective grounding
+ fuzzy relation grounding	Yes	Ablation	Tests whether retrieval can repair hallucinated relations
+ golden entity	Yes	Oracle ablation	Separates entity grounding from relation and KB coverage failures
ChatKBQA paper (Luo et al. 2024)	No	Reference only	Source-framework scale: WebQSP 79.8 F1 / 83.2 Hits@1 / 73.8 Acc

Baseline statement

The main baseline is the same-dataset generate-only pipeline on TempQuestions (Jia et al. 2018); ChatKBQA paper numbers are reference-only because they use different datasets.

Diagnostic Results

Stage	Value	Interpretation
Generation validity	87.6%	the model often emits structurally usable S-expressions
Grounded relation+entity pairs	5 / 995	most parser-valid predictions cannot be grounded in the KB
Questions with KB answers	1 / 268	execution rarely returns an answer
F1 / Hits@1 / Accuracy	0.0 / 0.0 / 0.0	answer-level quality is blocked by grounding and coverage

Key interpretation

The pipeline does not fail first at syntax generation. It fails at relation grounding, entity grounding, and KB execution coverage.

What Works vs. What Fails

Works

- end-to-end training and inference complete
- high valid S-expression diagnostic rate
- API/CLI/Streamlit/Docker surfaces exist
- evaluation isolates failure stages

Fails

- generated relations hallucinate beyond the KB
- entity IDs often point to the wrong entity
- narrow 22-relation training slice limits coverage
- raw answer-level score remains 0.0

Ablation takeaway

Fuzzy relation grounding raises relation match sharply, golden entities improve answer rate, but F1 remains zero; v2 needs retrieval-constrained generation plus broader relation/fact coverage.

Deployment Surfaces and Live Requirements

Primary surfaces

- FastAPI for programmatic inference
- CLI for reproducible local checks
- Docker for packaging
- Streamlit for presentation and inspection

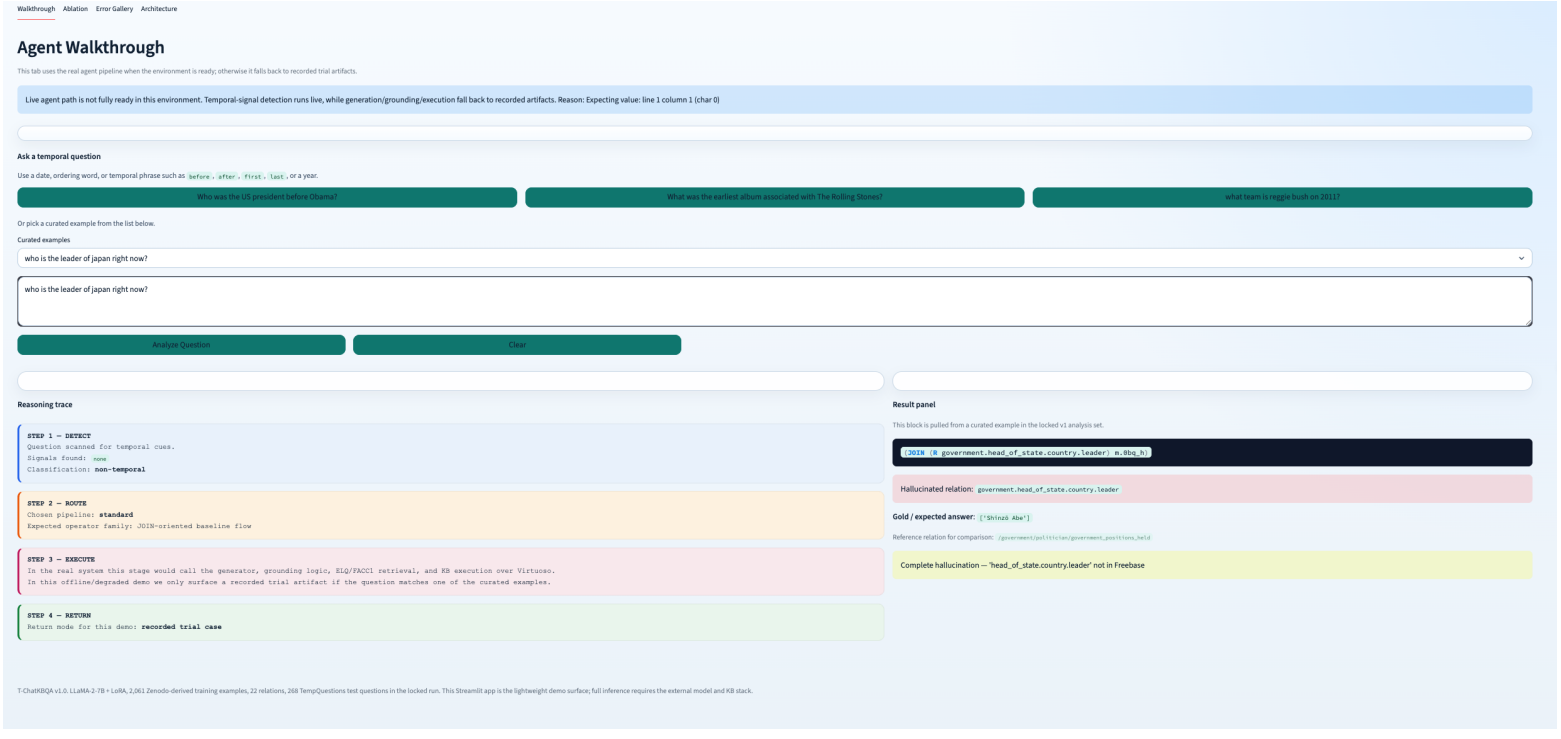
Full live requirements

- base LLaMA model
- LoRA adapter checkpoint
- Freebase/Virtuoso backend
- entity/relation retrieval assets

Demo policy

Missing assets trigger explicit degraded/offline mode instead of a fake live answer.

Streamlit Demo — Input, Output & Provenance



◀ **Input:** natural language question

◀ **Output:** generated S-expression + answer

◀ **Provenance:** temporal signals, routing, retries

Agentic Workflow

Agent steps

1. detect temporal signals
2. route to temporal or standard path
3. execute first attempt
4. retry with relaxed constraints if empty
5. return answer plus provenance

Why this is useful

The system exposes intermediate decisions instead of returning an opaque answer string.

```
~/Documents/Code/projects/ChatKBQA main +3 !13 ?37
❯ python3 -m src.cli --demo
============================================================
Agent Routing Demo (signal detection only)
============================================================

Q: Who was the US president before Obama?
   Signals found: ['before']
   -> Route: TEMPORAL pipeline

Q: What is the capital of France?
   Signals found: []
   -> Route: STANDARD pipeline

Q: Who held the position of US president during World War II?
   Signals found: ['during']
   -> Route: TEMPORAL pipeline

Q: What movies did Hitchcock direct in 1960?
   Signals found: ['in 1960']
   -> Route: TEMPORAL pipeline

Q: Name the first wife of Henry VIII.
   Signals found: ['first']
   -> Route: TEMPORAL pipeline

Q: Which country won the most gold medals at the 2008 Olympics?
   Signals found: ['2008']
   -> Route: TEMPORAL pipeline
============================================================
```

Visible provenance

- temporal signals
- generated logical form
- SPARQL when available
- retry count and reasoning trace

Monitoring, Privacy, and Robustness

Monitoring

- answer rate
- retry rate
- latency percentiles
- executor errors

Privacy

- public benchmark data
- minimize query logs
- avoid PII retention

Robustness

- OOD questions
- non-English inputs
- vague time wording
- missing KB facts

Responsible-use note

The system is more auditable than a free-form chatbot, but it inherits KB staleness, dataset bias, and model uncertainty.

Key Takeaways and Future Work

Submission-ready

- temporal KBQA architecture implemented
- data pipeline and tooling documented
- training/inference/evaluation completed
- deployable API/CLI/Streamlit surfaces

Next engineering priorities

- retrieval-constrained generation
- stronger entity linker integration
- broader relation/fact coverage
- benchmark regression gate before promotion

Final message

The v1 system is submission-ready as an engineering artifact, but not performance-ready as a production QA model.

References I

- Bollacker, K. et al. (2008). "Freebase: A Collaboratively Created Graph Database for Structuring Human Knowledge". In: *Proceedings of SIGMOD*, pp. 1247–1250.
- Hu, E. J. et al. (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *Proceedings of ICLR*.
- Jia, Z. et al. (2018). "TempQuestions: A Benchmark for Temporal Question Answering over Knowledge Bases". In: *Companion Proceedings of the Web Conference*.
- Luo, H. et al. (2024). "ChatKBQA: A Generate-then-Retrieve Framework for Knowledge Base Question Answering with Fine-tuned Large Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 2039–2056.

References II

 Touvron, H. et al. (2023). "Llama 2: Open Foundation and Fine-Tuned Chat Models". In: *arXiv preprint arXiv:2307.09288*.